Шинжлэх Ухаан Технологийн Их Сургууль

Мэдээлэл, Холбооны Технологийн Сургууль



F.CSM302 Үйлдлийн систем

БИЕ ДААЛТЫН ГАРЫН АВЛАГА

IV-2-8 · Scheduling Algorithms — хэрэглэгчийн ба operational заавар

Бие даалт №1

Шалгасан багш:	Г.Ганбат
Хуваарилалтын код:	IV-2-8 (IV өдөр · 2-р цаг · 8-р сэдэв)
Төсөл:	Scheduling Algorithms — Operating System Concepts (10th ed.), 5-р бүлэг, Programming Projects
UI дүрслэл:	Gantt chart (waiting, turnaround, response time-тай)
Гүйцэтгэсэн оюутан:	Л.Мөнхжин /B242270094/ B200930051 B221940051 B231940018 B241940025 B241940136
GitHub:	`https://github.com/Munkhjin12/fcsm302_hw`
Онлайн симулятор:	`https://fcsm302hw.vercel.app`

Улаанбаатар хот

2026 он

Гарчиг

1 Хэрэглэгчийн заавар **4**

1.1 Програмын зорилго 4
1.2 Дэмжих орчин 4
1.3 Нээх алхам 4
1.4 Онлайн хувилбар (Vercel) 4
1.5 Оролт ба тохиргоо 4
1.6 Ажиллуулах, зогсоох, reset 5
1.7 Дүрслэлийн тэмдэг, өнгө, төлөв 5
1.8 Жишээ 1. FCFS ба SJF-ийг харьцуулах 5
1.9 Жишээ 2. Round Robin ба quantum 5
1.10 Жишээ 3. C програмын бодит гаралтыг Gantt-аар харах 7
1.11 Түгээмэл алдаа ба шийдэл 7

2 Operational заавар **10**

2.1 Repository клонлох 10
2.2 OS, compiler/runtime, version 10
2.3 Dependency 10
2.4 Build ба Run 10
2.5 Үндсэн тестийн өгөгдөл 11
2.6 Нэмэлт boundary / error тест 11
2.7 Нэмэлт шалгалтууд 12
2.8 Known issues 12

Зургийн жагсаалт

1.1 Жишээ 1 — SJF-ийн үр дүн. 6
1.2 Жишээ 3 — C-ийн гаралтыг UI-д ачаалсан байдал. 8

Хүснэгтийн жагсаалт

1.1 Оролтын талбарууд 5
1.2 Дүрслэлийн тэмдэг, өнгө, төлөв 7
1.3 Түгээмэл алдаа ба шийдэл 9

2.1 Үндсэн тестийн хүлээгдсэн дундаж үзүүлэлт 11
2.2 Boundary ба error тест 12

Бичиг баримтын өөрчлөлтийн түүх

№	Засварласан агуулга	Тайлбар	Засварласан	Огноо
1	Бүх бүлэг	Зааврыг MUST-ийн LaTeX загварт шилжүүлэв.	Л.Мөнхжин	2026-10-27
2	Бүлэг 1, 2	Онлайн хувилбар (Vercel), GitHub холбоос, нэмэлт шалгалт (`reference_check.py`, `tid_race_test.c`) нэмэв; багшийн мэдээлэл, оюутнуудыг нөхөв.	Л.Мөнхжин	2026-10-29

Төслийн тайлбар толь

Нэр томъёо	Тайлбар
Code Viewer	Симулятор дахь код харуулах хэсэг: симуляторын горимд pseudo-code, «C-ийн гаралт оруулах» горимд C-ийн гаралтын мөрүүд.
Context switch	CPU-г нэг процессоос нөгөөд шилжүүлэх үйлдэл.
Idle	CPU сул байх үе (ямар ч процесс ирээгүй); Gantt дээр судалтай блок.
Quantum	Round Robin-д процесс нэг удаа үргэлжлүүлэн ажиллах дээд хугацаа (ms).
Ready queue	CPU хүлээж буй, ажиллахад бэлэн процессуудын дараалал.
Self-test	Симулятор доторх автомат шалгалт.
Timeline	Gantt chart-ын хэсэг: CPU мөр ба процесс бүрийн мөр.
WSL	Windows Subsystem for Linux — Windows дээр gcc/make ашиглах орчин.

Ерөнхий танилцуулга

Хэрэглэгчийн заавар (1-р бүлэг) нь вэб симулятор, operational заавар (2-р бүлэг) нь C хэрэгжилтийг клонлох, build хийх, тестлэхэд зориулагдсан.

1. Хэрэглэгчийн заавар

1.1 Програмын зорилго

CPU scheduling-ийн алгоритмуудыг (FCFS, SJF, SRTF, Priority, Priority-preemptive, Round Robin, Priority+RR) өөр өгөгдөл, параметрээр ажиллуулж, Gantt chart болон waiting / turnaround / response time-аар харьцуулах.

1.2 Дэмжих орчин

- Орчин үеийн хөтөч (Chrome, Edge, Firefox, Safari). Интернет, суулгалт шаардлагагүй. Шалгасан: Chromium 141.
- Утас, таблет дээр ажиллана (нарийн дэлгэц дээр блокууд доош давхарлагдана).

1.3 Нээх алхам

1. Repository-г татаж задална (`https://github.com/Munkhjin12/fcsm302_hw`), эсвэл онлайн хувилбарыг шууд нээнэ: `https://fcsm302hw.vercel.app`.
2. `src/web/index.html`-ийг хөтөч рүү чирж нээнэ (эсвэл давхар товшино). Нээхэд номын жишээ (T1…T5) автоматаар ачаалагдана.

1.4 Онлайн хувилбар (Vercel)

Симулятор нь гадны сангүй нэг `index.html` тул Vercel-д статик сайтаар байршуулж болно: repository-г GitHub-д push хийж, Vercel дээр *Add New* → *Project* сонгоод repository-г холбоно. Repository-ийн язгуурт байгаа `vercel.json` нь гаралтын хавтсыг `src/web` гэж заасан тул Framework = *Other*, Build/Install command хоосон байлгаад *Deploy* дарна. Энэ төслийн онлайн хувилбар: `https://fcsm302hw.vercel.app` (суулгах зүйлгүй, хөтөч дээр шууд нээнэ).

1.5 Оролт ба тохиргоо

Товчнууд: **+ Процесс** мөр нэмнэ; × устгана; **Номын жишээ** / **Arrival-тай** / **RR жишээ** бэлэн өгөгдөл ачаална; **Reset** анхны төлөвт буцаана; «schedule.txt оруулах» хэсэгт `name, priority, burst` мөрүүдийг буулгаад **Ачаалах** дарна (arrival = 0).

Хүснэгт 1.1. Оролтын талбарууд

Талбар	Утга	Тайлбар
Нэр	давтагдахгүй, хоосон биш	Gantt дээр харагдана
Arrival	бүхэл ≥ 0	Номын жишээнд бүгд 0
Burst	бүхэл, 1…1000	CPU burst (ms)
Priority	бүхэл, 1…10	Их тоо = өндөр priority
Quantum	бүхэл ≥ 1	Зөвхөн Round Robin, Priority+RR-д идэвхтэй
Процессын тоо	1…20	

1.6 Ажиллуулах, зогсоох, reset

Ажиллуулах дарахад Timeline, Метрик, pseudo-code гарна. **Тоглуулах** нь 150 ms тутамд 1 ms-ийн алхам урагшлуулна, дахин дарж **Зогсоох**. Slider, ◄ ► товчоор гараар алхамчилна, эхлэл/төгсгөл рүү үсрэх товчоор тус тус эхлэл, төгсгөл рүү үсэрнэ. Алгоритм эсвэл өгөгдөл солиход үр дүн цэвэрлэгдэнэ — дахин **Ажиллуулах** дарна. **Reset** нь бүгдийг буцаана.

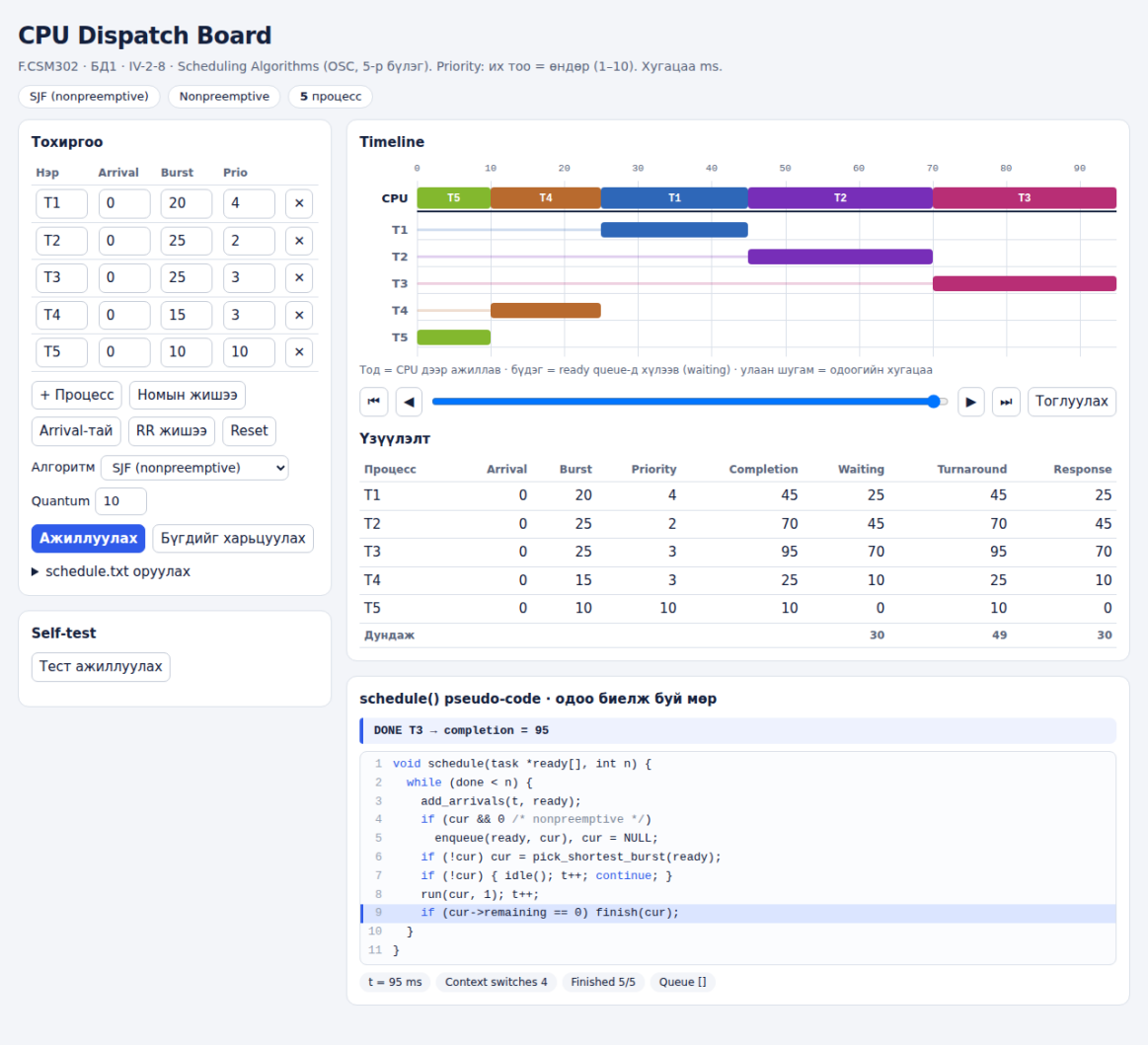
1.7 Дүрслэлийн тэмдэг, өнгө, төлөв

1.8 Жишээ 1. FCFS ба SJF-ийг харьцуулах

Номын жишээг ачаалж (**Номын жишээ**), алгоритмыг *FCFS* болгон **Ажиллуулах**: дундаж waiting 44. Дараа нь *SJF*: дундаж waiting 30, T5 (burst 10) эхэнд ажиллана. Эсвэл **Бүгдийг харьцуулах** дарж бүгдийг нэг дор үзнэ.

1.9 Жишээ 2. Round Robin ба quantum

Алгоритм = *Round Robin*, Quantum = 10 $\rightarrow$ дундаж response 20, waiting 57. Quantum-ыг 4 болгон дахин ажиллуулбал response 8, waiting 60.4 болно. Slider-ийг чирж task бүр хэзээ ээлжээр CPU авч байгааг ажиглана.



Зураг 1.1. Жишээ 1 — SJF-ийн үр дүн.

Хүснэгт 1.2. Дүрслэлийн тэмдэг, өнгө, төлөв

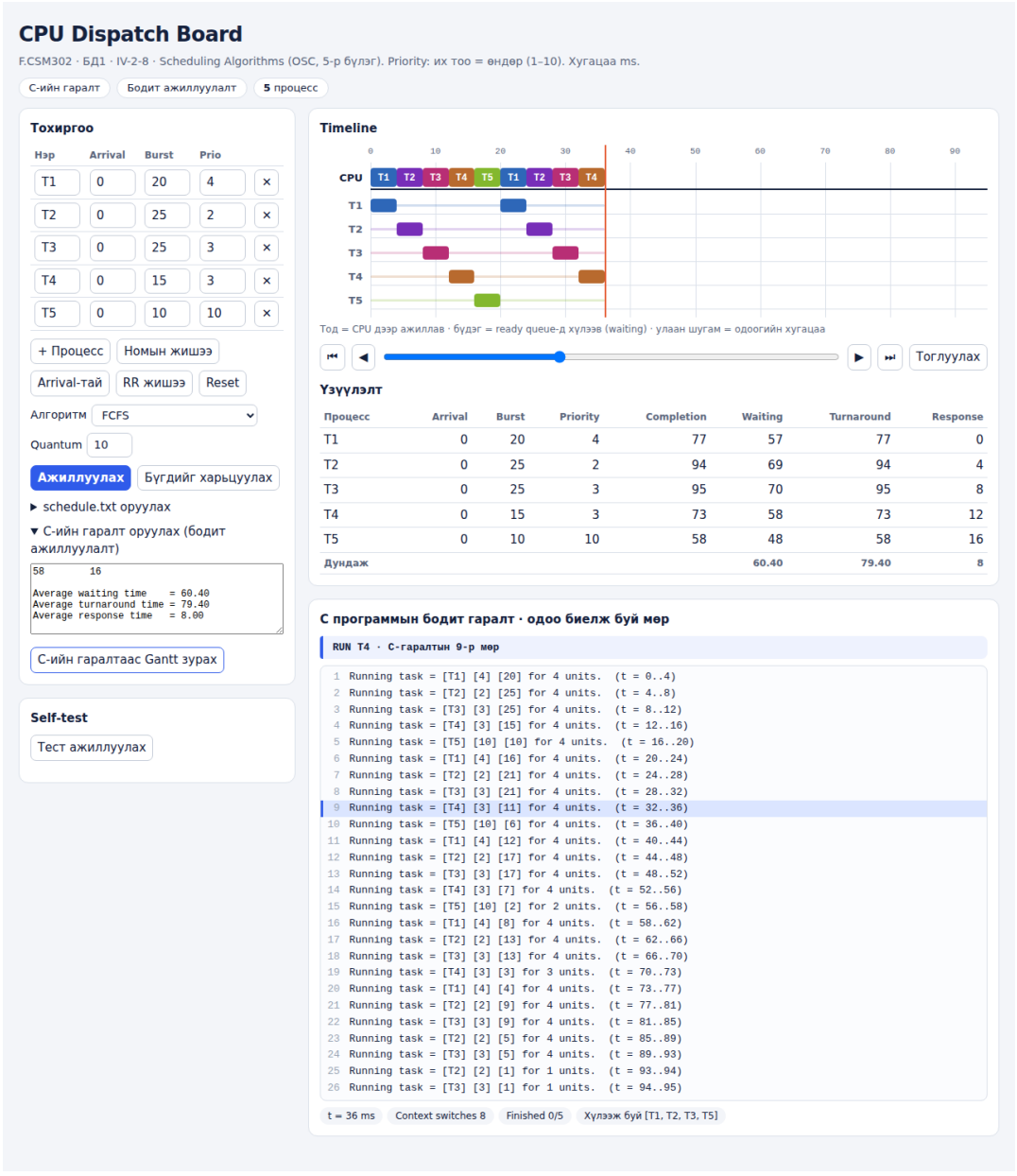
Тэмдэг	**Утга**
CPU мөр	Тухайн цагт CPU дээр ажиллаж буй процесс (өнгө = процесс)
Процессын мөрийн тод блок	Тухайн процесс CPU дээр ажилласан хугацаа
Процессын мөрийн бүдэг шугам	Ready queue-д хүлээсэн хугацаа (waiting)
Судалтай блок (idle)	CPU сул (ямар ч процесс ирээгүй)
Улаан босоо шугам	Одоогийн хугацаа t
Цэнхэр pill	Сүүлийн үйлдэл: PICK / RUN / PREEMPT / DONE / IDLE
Pseudo-code-ийн тодорсон мөр	Тухайн алхамд биелсэн мөр
Доод мөрийн chip-үүд	t, context switch, дууссан тоо, ready queue

1.10 Жишээ 3. C програмын бодит гаралтыг Gantt-аар харах

1. C хэсэгт `./rr ../../tests/schedule.txt 4` ажиллуулна. macOS дээр гаралтыг шууд хуулж болно: `./rr ../../tests/schedule.txt 4 | pbcopy` (эсвэл Terminal-аас гараар сонгож хуулна).
2. Вэб симуляторт «C-ийн гаралт оруулах» хэсгийг нээж буулгана.
3. «C-ийн гаралтаас Gantt зурах» дарна.

Gantt, метрик болон Code Viewer (C-ийн мөрүүд) C-ийн бодит гаралтаас зурагдаж, дундаж нь C-ийн хэвлэсэнтэй таарна.

1.11 Түгээмэл алдаа ба шийдэл



Зураг 1.2. Жишээ 3 — С-ийн гаралтыг UI-д ачаалсан байдал.

Хүснэгт 1.3. Түгээмэл алдаа ба шийдэл

Мэдээлэл / байдал	**Шалтгаан → Шийдэл**
«Quantum нь 1 ба түүнээс их бүхэл тоо байх ёстой»	Quantum 0, сөрөг эсвэл хоосон → 1 ба түүнээс их бүхэл тоо оруулна
«…arrival / burst / priority … байх ёстой»	Тухайн мөрийн утга хүрээнээс гадуур → мөрийн дугаарыг уншиж засна
«… нэр давхцсан»	Хоёр процесс ижил нэртэй → нэрийг өөрчилнө
«Импорт амжилтгүй»	`schedule.txt`-ийн мөр *name, priority, burst* хэлбэртэй биш → таслал, тоог шалгана
Үр дүн харагдахгүй	Алгоритм/өгөгдөл солисны дараа **Ажиллуулах** дараагүй → дарна
«C-ийн гаралтыг уншиж чадсангүй»	Гаралт бүрэн биш эсвэл өөр програмын гаралт → `./rr …` гэх мэт бүтэн гаралтыг (`Running task` мөрүүд) буулгана. Хугацаа тасралтгүй байх ёстой
Self-test FAIL	Кодыг өөрчилсөн байж болзошгүй → эх хувилбараар сольж шалгана

2. Operational заавар

2.1 Repository клонлох

```
git clone https://github.com/Munkhjin12/fcsm302_hw.git
cd fcsm302_hw
```

Хөрвүүлсэн файлууд (`fcfs`, `sjf`, …) `.gitignore`-д орсон тул repo-д байхгүй — `make`-ээр үүснэ.

2.2 OS, compiler/runtime, version

Шалгасан орчин: macOS (Terminal/zsh, gcc = Apple clang, Safari) — баг гараар; Linux (gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0; Node.js v22.22.2; Linux 6.18.44-fc-v153 x86_64; Chromium 141.0.7390.37) — автомат. C хэсэгт gcc (C11 дэмжсэн), make, sh хэрэгтэй: Linux, macOS (Xcode CLT) эсвэл Windows дээр WSL. Windows-ын цэвэр CMD/PowerShell дээр `__sync_fetch_and_add`, `strdup`-ийн улмаас MinGW/WSL ашиглана.

2.3 Dependency

Гадны сан шаардлагагүй. Ubuntu/Debian: `sudo apt install build-essential`. macOS: `xcode-select --install`.

2.4 Build ба Run

```
cd src/c
make
make fcfs
./fcfs ../../tests/schedule.txt
./rr ../../tests/schedule.txt 4
make clean
```

Тайлбар: `make` — бүх алгоритмыг (`fcfs sjf priority rr priority_rr`) хөрвүүлнэ; `make fcfs` — зөвхөн нэгийг; `./rr … 4` дахь сүүлийн аргумент нь quantum = 4 (заавал биш, анхны утга 10).

2.5 Үндсэн тестийн өгөгдөл

`tests/schedule.txt` (номын жишээ): `T1,4,20` / `T2,2,25` / `T3,3,25` / `T4,3,15` / `T5,10,10`. Хүлээгдсэн дундаж (waiting / turnaround / response):

Хүснэгт 2.1. Үндсэн тестийн хүлээгдсэн дундаж үзүүлэлт

Алгоритм	**Команд**	**Waiting**	**Turnaround**	**Response**
FCFS	`./fcfs ../../tests/schedule.txt`	44.00	63.00	44.00
SJF	`./sjf ../../tests/schedule.txt`	30.00	49.00	30.00
Priority	`./priority ../../tests/schedule.txt`	33.00	52.00	33.00
Round Robin (q=10)	`./rr ../../tests/schedule.txt`	57.00	76.00	20.00
Priority + RR (q=10)	`./priority_rr ../../tests/schedule.txt`	35.00	54.00	30.00

Автомат тест: `make test` (эсвэл `sh tests/run_tests.sh`). Хүлээгдсэн гаралт: `passed: 18, failed: 0`.

2.6 Нэмэлт boundary / error тест

Хүлээгдсэн гаралтын жишээ (FCFS):

```
Running task = [T1] [4] [20] for 20 units.  (t = 0..20)
Running task = [T2] [2] [25] for 25 units.  (t = 20..45)
Running task = [T3] [3] [25] for 25 units.  (t = 45..70)
Running task = [T4] [3] [15] for 15 units.  (t = 70..85)
Running task = [T5] [10] [10] for 10 units.  (t = 85..95)

Task     tid  Pri  Burst   Waiting  Turnaround   Response
T1         1    4     20         0          20          0
T2         2    2     25        20          45         20
T3         3    3     25        45          70         45
T4         4    3     15        70          85         70
T5         5   10     10        85          95         85
Average waiting time    = 44.00
Average turnaround time = 63.00
Average response time   = 44.00
```

Хүснэгт 2.2. Boundary ба error тест

Команд	**Хүлээгдсэн үр дүн**
`./priority_rr ../../tests/schedule_ties.txt 4`	avg waiting 12.75, turnaround 20.50, response 8.50 (A, B, C ээлжилнэ)
`./fcfs ../../tests/bad_priority.txt`	«priority 11 is outside 1..10», exit code $\neq 0$
`./fcfs ../../tests/bad_burst.txt`	«CPU burst must be at least 1», exit $\neq 0$
`./fcfs ../../tests/bad_format.txt`	«expected name, priority, burst», exit $\neq 0$
`./fcfs ../../tests/bad_empty.txt`	«contains no tasks», exit $\neq 0$
`./rr ../../tests/schedule.txt 0`	«quantum must be an integer from 1 to 1000», exit $\neq 0$
`./fcfs missing.txt`	файл нээж чадсангүй гэсэн алдаа, exit $\neq 0$

2.7 Нэмэлт шалгалтууд

```
# independent Python model: 20 combinations, prints "mismatches: 0"
make -C src/c
python3 tests/reference_check.py

# atomic tid test: plain x = x + 1 vs __sync_fetch_and_add
gcc -O2 -pthread tests/tid_race_test.c -o tid_race_test
./tid_race_test
```

`tid_race_test`-ийн хүлээгдэх гаралт: «__sync_fetch_and_add» мөрөнд `lost updates: 0`. Энгийн `x = x + 1` мөрөнд update алдах нь *санамсаргүй* тул зарим ажиллуулалтад 0 гарч болно; хэд хэдэн удаа давтана.

2.8 Known issues

- C хувилбар бүх task $t = 0$ үед ирнэ гэж үзнэ (номын нөхцөл); arrival, preemptive хувилбарууд зөвхөн вэб симулятор дээр.
- Context switch-ийн зардал 0 ms гэж үзсэн.
- Симулятор: burst ≤ 1000, ≤ 20 процесс; симуляторын горимд Code Viewer нь pseudo-

code (C-гаралтын горимд C-ийн мөр).

- Task нэр давхцахыг C хувилбар шалгадаггүй (симулятор шалгана).
- Task-уудын бүртгэл (`all_tasks[]`) олон thread-д аюулгүй биш; атомар болгосон нь зөвхөн tid.
- Симулятор процессын нэрийг HTML-д шууд оруулдаг тул тусгай тэмдэгттэй нэр (жишээ нь `<` эсвэл `"`) оруулбал дүрслэл эвдрэх боломжтой (зөвхөн өөрийн хөтөч дээр).
- Windows-ын цэвэр MSVC компайлер дэмжигдэхгүй (gcc/WSL ашиглана).

AI ашигласан тухай

Энэ заавар болон төслийн кодын эхний хувилбар, тест, баримт бичгийн ноорог бичихэд Claude (Anthropic) ашиглагдсан (Тайлангийн «AI ашигласан тухай» хэсэгтэй адил). Демо видеоны хоолойг ElevenLabs (AI дуу хоолой)-оор үүсгэсэн. Багийнхан зааврын алхмуудыг өөрсдөө ажиллуулж шалгасан.